

PONTIFICIA
UNIVERSIDAD
CATÓLICA
DE CHILE

Procesamiento de Lenguaje Natural

github.com/marcelomendoza/IIC3670

Marcelo Mendoza

Lo que guarda el optimizador

AdamW guarda por cada parámetro dos promedios de sus gradientes, m y v, y actualiza una copia de los pesos en 32 bits.

$$g_t = \nabla_\theta \mathcal{L}(\theta_{t-1}) \qquad \text{gradiente del lote } t$$

$$m_t = \beta_1\, m_{t-1} + (1-\beta_1)\, g_t \qquad \text{primer momento: promedio móvil de } g,\ \beta_1{=}0{,}9$$

$$v_t = \beta_2\, v_{t-1} + (1-\beta_2)\, g_t^2 \qquad \text{segundo momento: promedio móvil de } g^2,\ \beta_2{=}0{,}999$$

$$\hat{m}_t = m_t/(1-\beta_1^t), \qquad \hat{v}_t = v_t/(1-\beta_2^t) \qquad \text{corrigen el sesgo hacia 0 de los primeros pasos}$$

$$\theta_t = \theta_{t-1} - \eta\left(\hat{m}_t/\left(\sqrt{\hat{v}_t}+\epsilon\right) + \lambda\,\theta_{t-1}\right) \qquad \eta\text{: tasa de aprendizaje; } \lambda\text{: weight decay}$$

Primer momento m

Promedio móvil de los gradientes, estima su media E[g]. Da la dirección del paso.

Segundo momento v

Promedio móvil de los gradientes al cuadrado, estima $E[g^2]$. Dividir por su raíz deja un paso cercano a η en cada parámetro.

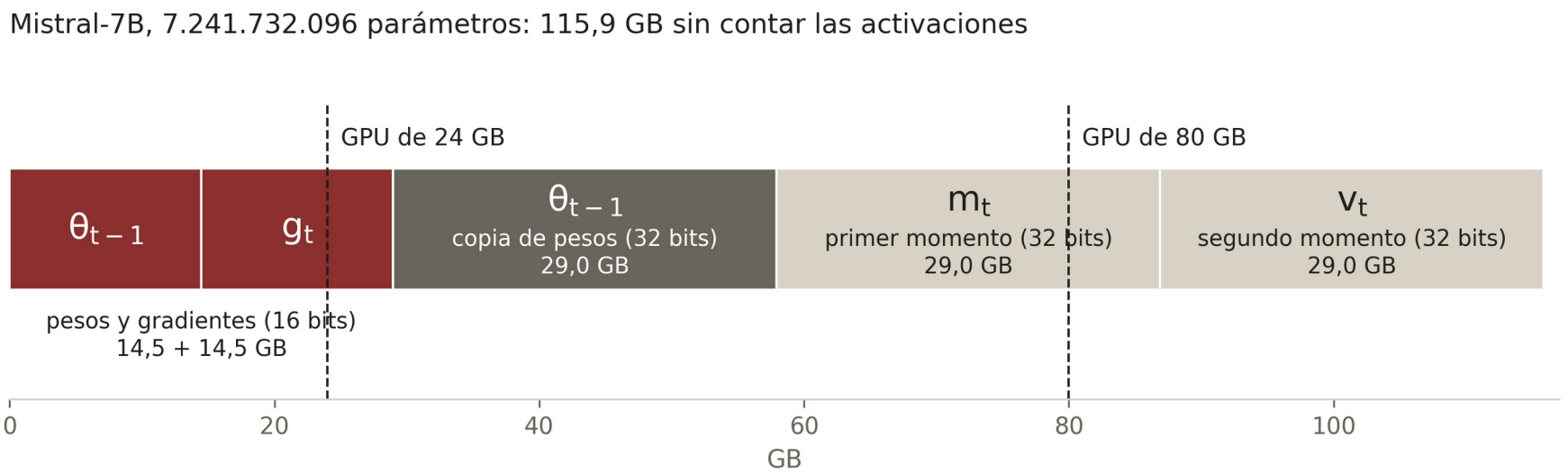
Copia en 32 bits

En 16 bits, 0,02 + 0,00002 se redondea a 0,02 y el paso se pierde. La actualización se hace en una copia en 32 bits.

Kingma y Ba, ICLR 2015 (Adam); Loshchilov y Hutter, ICLR 2019 (AdamW); Micikevicius et al., ICLR 2018 (precisión mixta).

El costo del ajuste completo

Ajustar todos los pesos requiere guardar en la GPU los pesos, sus gradientes y el estado del optimizador: unos 16 bytes por parámetro, y una copia completa del modelo por cada tarea.



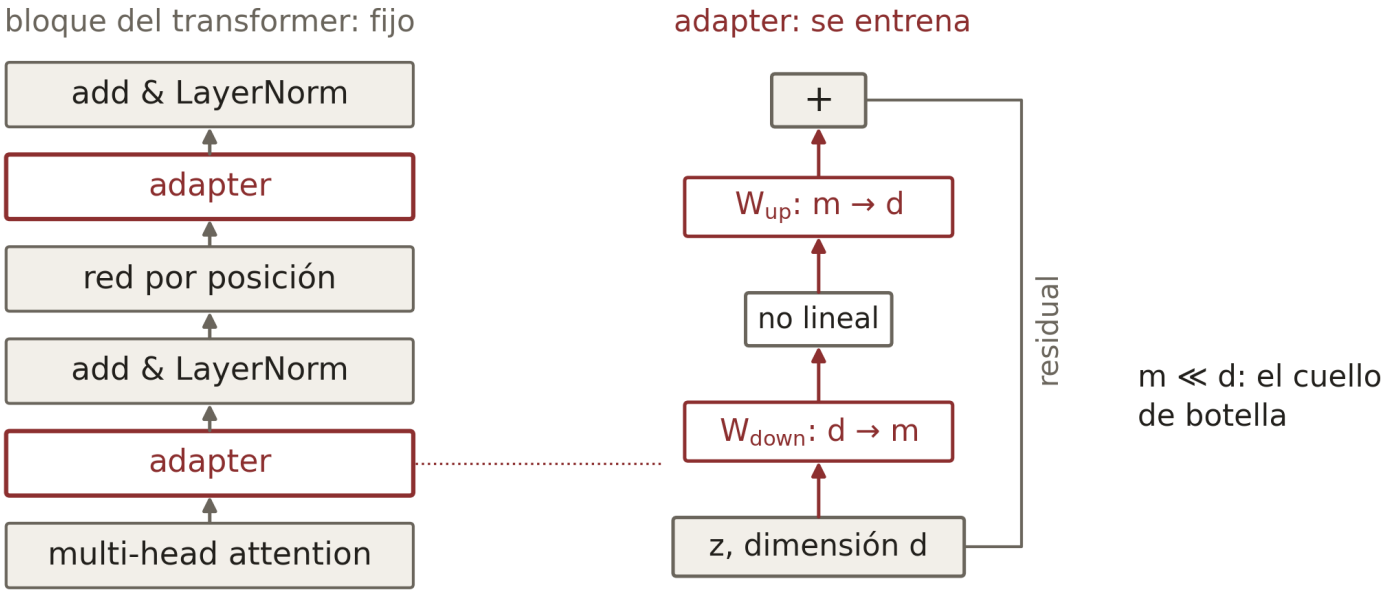
Configuración publicada de Mistral-7B; Rajbhandari et al., SC 2020 (ZeRO).

Adapters

Se agregan capas pequeñas dentro de cada bloque y solo esas se entrenan. Cada adapter proyecta a una dimensión m mucho menor que d y vuelve, con un residual.

$$z' = z + \boldsymbol{W_{\text{up}}}\,\sigma\left(\boldsymbol{W_{\text{down}}}\,z\right) \qquad W_{\text{down}} \in \mathbb{R}^{m\times d},\ W_{\text{up}} \in \mathbb{R}^{d\times m},\ m \ll d$$

$$\text{parámetros} = 2md + d + m \qquad \text{por adapter; se entrenan solo los adapters}$$



Houlsby et al., ICML 2019; Li y Liang, ACL 2021.

La dimensión intrínseca del ajuste

Un modelo preentrenado se puede ajustar optimizando pocos parámetros en un subespacio aleatorio. El ajuste necesita muchos menos grados de libertad que los pesos que tiene el modelo.

$$\theta = \theta_0 + P\,\theta_d \qquad \theta, \theta_0 \in \mathbb{R}^D\text{: todos los pesos; } \theta_0 \text{ preentrenados, fijos}$$

$$P \in \mathbb{R}^{D \times d}, \qquad d \ll D \qquad \text{proyección aleatoria fija; solo } \theta_d \in \mathbb{R}^d \text{ se entrena}$$

El experimento

Todos los pesos θ se escriben como los preentrenados θ_0 más una proyección aleatoria P, fija, de un vector θ_d de dimensión d. Solo θ_d se entrena.

El resultado

Con d = 200, RoBERTa llega al 90 % del desempeño del ajuste completo en MRPC, una tarea de paráfrasis de GLUE.

La consecuencia

Si basta un subespacio chico, la actualización de los pesos debería poder escribirse con pocos parámetros: es la hipótesis de LoRA.

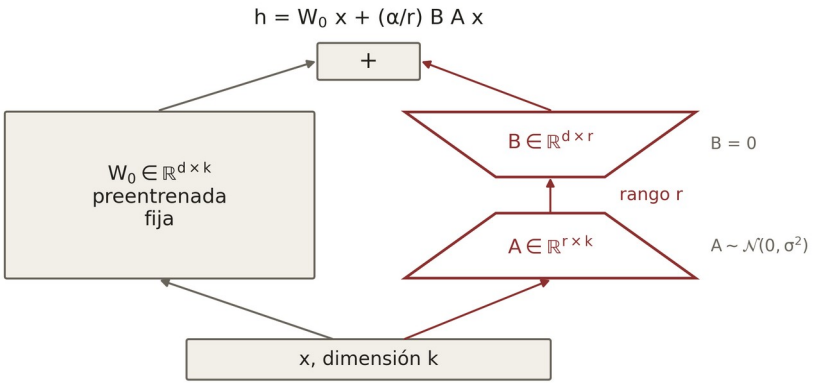
Aghajanyan, Zettlemoyer y Gupta, ACL 2021.

LoRA

Low-Rank Adaptation: la matriz preentrenada W_0 queda fija y su actualización se escribe como el producto BA de dos matrices delgadas, de rango r. Solo B y A se entrenan.

$$W = W_0 + \Delta W = W_0 + \boldsymbol{BA}, \qquad r \ll \min(d, k) \qquad W_0 \in \mathbb{R}^{d\times k} \text{ fija; } B \in \mathbb{R}^{d\times r},\ A \in \mathbb{R}^{r\times k}$$

$$A \sim \mathcal{N}(0, \sigma^2), \qquad B = 0 \qquad \text{al inicio } \Delta W = 0\text{: se parte del modelo preentrenado}$$



A aleatoria

La entrada de B es Ax. Si A = 0, B recibe una entrada nula y no aprende. Al azar, cada fila de A mira una dirección distinta de x.

B en cero

BA = 0 y el modelo parte igual al preentrenado. Al principio solo B aprende; el error llega a A a través de B cuando B deja de ser cero.

Elegir r

Es un hiperparámetro. Se usa entre 8 y 64, más alto si la tarea se aleja del preentrenamiento.

Hu et al., ICLR 2022; Aghajanyan et al., ACL 2021; Hayou et al., NeurIPS 2024; Biderman et al., TMLR 2024.

La escala α/r

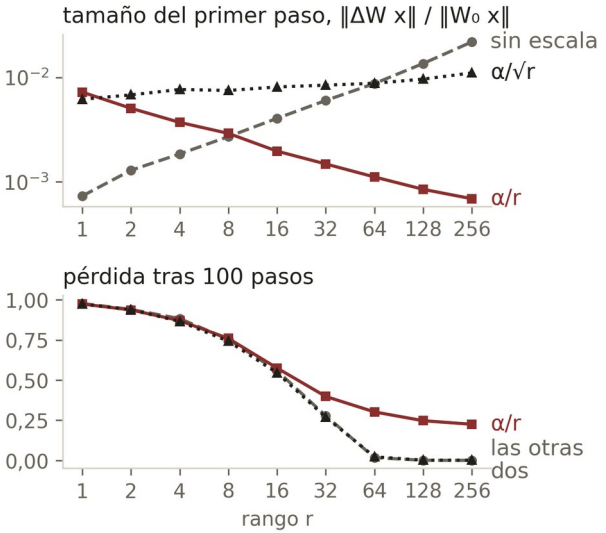
BAx es una suma de r términos, y con Adam cada uno se mueve lo mismo. Sin una escala, la actualización crece con el rango; α/r la compensa para no reajustar η.

$$BA\,x = \sum_{j=1}^{r} b_j\,(a_j^\top x) \qquad \text{una suma de } r \text{ términos: columna } j \text{ de } B\text{, fila } j \text{ de } A$$

$$|\Delta B_{ij}| \approx \eta \qquad \text{con Adam, cada entrada se mueve cerca de } \eta\text{, sea cual sea su gradiente}$$

$$h = W_0\,x + \tfrac{\alpha}{r}\,BA\,x \qquad \text{Hu et al.: compensa los } r \text{ términos; } \alpha \text{ se fija una vez}$$

$$h = W_0\,x + \tfrac{\alpha}{\sqrt{r}}\,BA\,x \qquad \text{rsLoRA: } 1/r \text{ compensa de más cuando } r \text{ es grande}$$



Sin escala

De r = 1 a r = 256 el primer paso crece 30 veces con el mismo η.

α/r

Es la escala de Hu et al. Con r grande compensa de más, el paso se achica y el ajuste aprende más lento.

α/√r

Es la corrección de rsLoRA. Mantiene el paso casi constante y aprovecha el rango extra.

Hu et al., ICLR 2022; Kalajdzievski, 2023 (rsLoRA). Curvas: una capa de 1.024 × 1.024 entrenada con Adam.

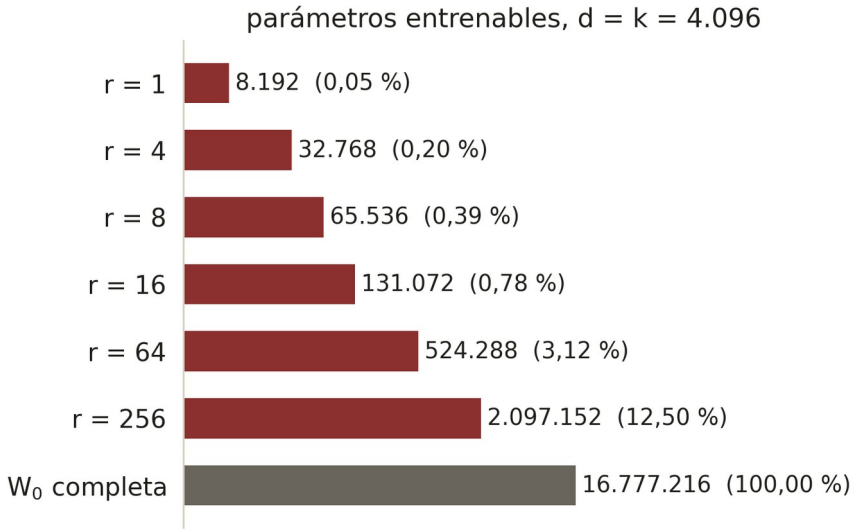
LoRA en el forward y en la inferencia

Al entrenar, el forward suma la salida de W_0 y la de BA. Al terminar, BA se suma a W_0 una sola vez, y la inferencia cuesta lo mismo que la del modelo original.

$$h = W_0\,x + \tfrac{\alpha}{r}\,BA\,x \qquad \alpha\text{: constante de escala; } \alpha/r \text{ evita reajustar la tasa al cambiar } r$$

$$W = W_0 + \tfrac{\alpha}{r}\,BA \qquad \text{inferencia: se fusiona una vez, sin costo extra por token}$$

$$r\,(d+k) \ll d\,k \qquad \text{parámetros entrenables contra los de } W_0$$



Escala α/r

La de la lámina anterior. Se aplica en el entrenamiento y queda dentro de W al fusionar.

Fusión

$W = W_0 + (\alpha/r)BA$: misma forma que W_0, sin cómputo extra por token.

Una tarea, un adaptador

Se guardan solo B y A; se cambia de tarea restando un BA y sumando otro.

Hu et al., ICLR 2022.

LoRA en la práctica

Con r = 4 y solo W_q y W_v adaptadas, LoRA iguala o supera al ajuste completo en las tareas del paper. Un rango tan bajo como 1 basta en varias de ellas.

En GPT-3, 175.000 millones de parámetros	Ajuste completo	LoRA
parámetros entrenables	todos	10.000 veces menos
memoria de GPU al entrenar	1,2 TB	350 GB
tamaño de lo que se guarda por tarea	350 GB	35 MB
100 tareas en disco	≈35 TB	≈354 GB
velocidad de entrenamiento		25 % más rápido

Hu et al., ICLR 2022.

Cuantización

LoRA reduce lo que se entrena, pero el modelo congelado sigue en memoria. Cuantizar es guardar cada peso con menos bits, por ejemplo 4 en lugar de 16, y recuperar un valor aproximado al usarlo.

$$s = \max_j |w_j|, \qquad \hat{w}_j = w_j / s \in [-1, 1] \qquad \text{una escala por bloque de pesos}$$

$$c_j = \arg\min_i \left| \hat{w}_j - \ell_i \right| \qquad \ell_0, \ldots, \ell_{15}\text{: los 16 niveles; se guarda } c_j \text{ en 4 bits}$$

$$w_j \approx s\, \ell_{c_j} \qquad \text{decuantizar: se recupera un valor aproximado}$$

La escala

Cada bloque de pesos se divide por su mayor valor absoluto s, y queda en [−1, 1].

El código

Con 4 bits hay 16 niveles. Se guarda el índice del nivel más cercano: 0,23456 queda en el nivel 9, 0,2, código 1001.

Decuantizar

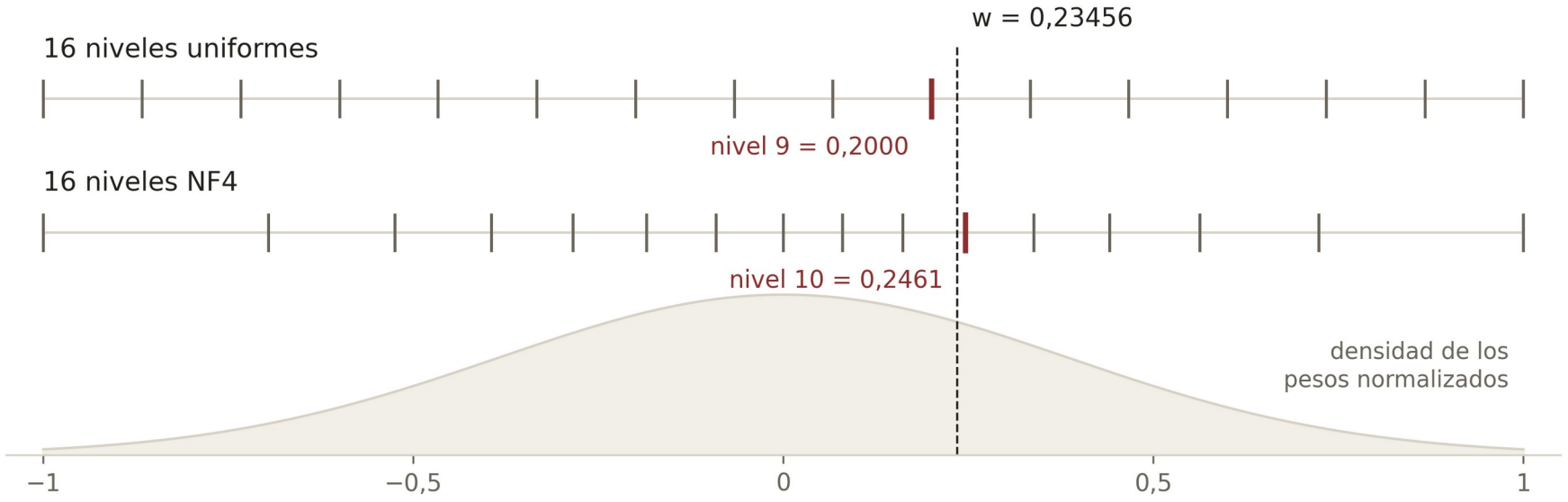
Para usar el peso se multiplica el nivel por s. Se pierde la diferencia, aquí 0,03456.

Dettmers et al., NeurIPS 2023.

NormalFloat de 4 bits

Los pesos de una red tienen una distribución cercana a la normal. NF4 pone los 16 niveles en los cuantiles de una normal: más niveles donde hay más pesos.

$$\ell_i = \tfrac{1}{2}\left(Q\left(\tfrac{i}{2^k+1}\right) + Q\left(\tfrac{i+1}{2^k+1}\right)\right) \qquad Q\text{: cuantil de } \mathcal{N}(0,1)\text{; después se reescalan a } [-1,1]$$



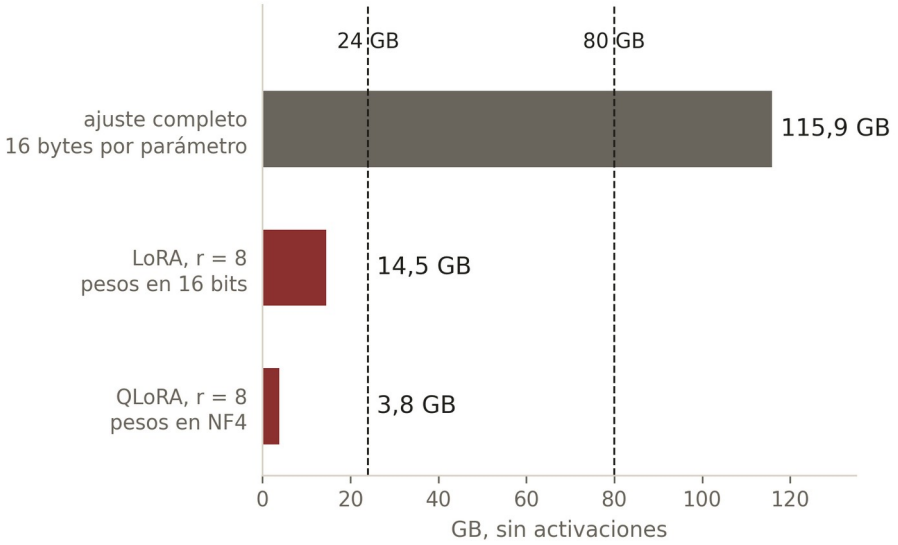
Dettmers et al., NeurIPS 2023. Niveles de la biblioteca bitsandbytes.

QLoRA

El modelo congelado se guarda en NF4 y LoRA se entrena encima, en 16 bits. Cada peso se decuantiza a BF16 justo antes de multiplicar.

$$h = \text{decuantizar}\left(s, \boldsymbol{W_0^{\text{NF4}}}\right) x + \tfrac{\alpha}{r}\, \boldsymbol{B A}\, x \qquad \text{el producto se calcula en BF16}$$

W_0^{NF4} fijos, 4 bits; B, A en BF16, se entrenan — el gradiente pasa por W_0 pero no la actualiza



BF16

Brain float: 1 bit de signo, 8 de exponente y 7 de mantisa. El rango de FP32, con menos precisión.

Memoria

Un modelo de 65.000 millones baja de 780 GB a menos de 48 GB.

Calidad

Iguala al ajuste completo en 16 bits. Guanaco llega al 99,3 % de ChatGPT en el benchmark de Vicuna.

Paged optimizers

Si falta memoria en la GPU, el estado del optimizador pasa a la del CPU.

Dettmers et al., NeurIPS 2023. Memoria de Mistral-7B calculada sin activaciones.

Mixture of Experts

Otra forma de eficiencia: un modelo con muchos más parámetros que los que usa cada token. La red se divide en expertos y cada token pasa solo por algunos.

$$p(x) = \mathrm{softmax}(W_r\, x) \in \mathbb{R}^N$$ router: $p_e(x) \geq 0$ es el peso del experto e para x; suman 1

$$f(x) = \sum_{e=1}^{N} p_e(x)\, E_e(x)$$ densa: se evalúan los N expertos

$$f(x) = \sum_{e \in \mathcal{T}(x)} g_e(x)\, E_e(x), \qquad g_e = \frac{p_e}{\sum_{e' \in \mathcal{T}(x)} p_{e'}}$$ dispersa: solo los k con mayor p_e, con sus pesos renormalizados

El router

Una capa lineal W_r con una softmax. Para cada entrada x da N pesos $p_e(x)$, positivos y que suman 1: cuánto pesa cada experto.

Densa

La salida es la suma de los N expertos E_e, cada uno multiplicado por su peso. Se evalúan todos.

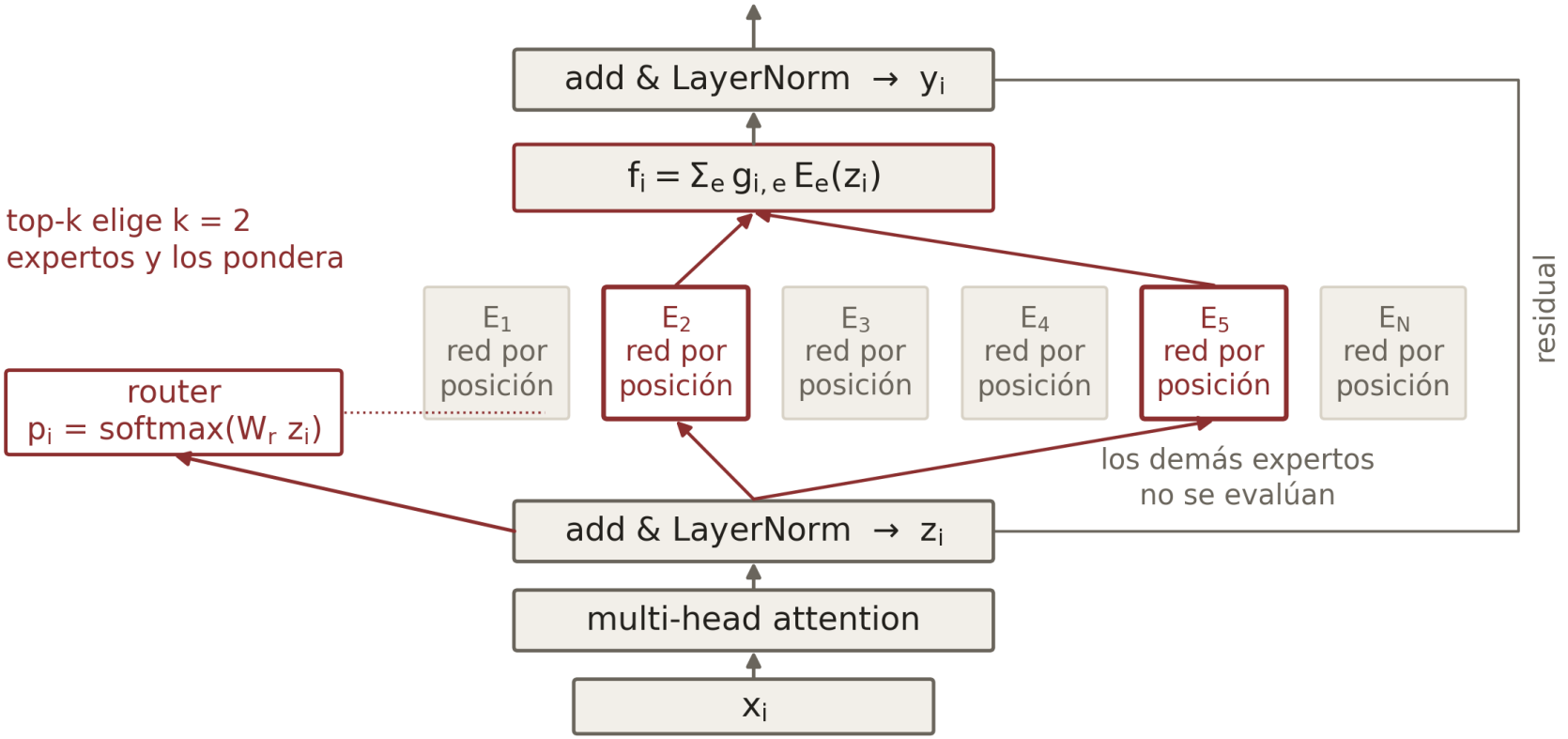
Dispersa

Solo se evalúan los k expertos con mayor peso, y sus pesos g se renormalizan para sumar 1. El cómputo depende de k y no de N.

Jacobs, Jordan, Nowlan y Hinton, Neural Computation 1991; Shazeer et al., ICLR 2017.

Un bloque con MoE

La red por posición del bloque se reemplaza por N expertos, cada uno con su propia red por posición, y un router que elige k de ellos para cada token. La atención no cambia.



Lepikhin et al., ICLR 2021; Fedus, Zoph y Shazeer, JMLR 2022.

Ecuaciones de transferencia

Son las del bloque del transformer con una sola línea cambiada: f_i deja de ser una red por posición y pasa a ser la suma ponderada de los k expertos que elige el router.

$$a_i = \mathrm{MultiHead}\big(x_i, [x_1, \ldots, x_j]\big), \quad j \le i$$ atención causal, sin cambios

$$z_i = \mathrm{LayerNorm}(x_i + a_i)$$ residual y normalizar

$$s_i = \boldsymbol{W_r}\, z_i, \qquad p_i = \mathrm{softmax}(s_i)$$ router: $W_r \in \mathbb{R}^{N \times d}$, una probabilidad por experto

$$\mathcal{T}_i = \mathrm{topk}(p_i, k)$$ los k expertos con mayor probabilidad

$$g_{i,e} = p_{i,e} \Big/ \textstyle\sum_{e' \in \mathcal{T}_i} p_{i,e'}$$ pesos renormalizados sobre los elegidos

$$E_e(z) = W_2^{(e)} \max\big(0,\, W_1^{(e)} z\big)$$ cada experto es una red por posición

$$f_i = \sum_{e \in \mathcal{T}_i} \boldsymbol{g_{i,e}}\, E_e(z_i)$$ reemplaza a $f_i = W_2 \max(0, W_1 z_i)$

$$y_i = \mathrm{LayerNorm}(z_i + f_i)$$ salida del bloque

Fedus, Zoph y Shazeer, JMLR 2022; Jiang et al., 2024 (Mixtral).

Un token por el router

Con cuatro expertos y k = 2: el router da un puntaje s a cada experto, la softmax los vuelve probabilidades p, se eligen los dos mayores y sus pesos g se renormalizan para sumar uno.

$$p(x) = \mathrm{softmax}(W_r\, x) \in \mathbb{R}^N$$

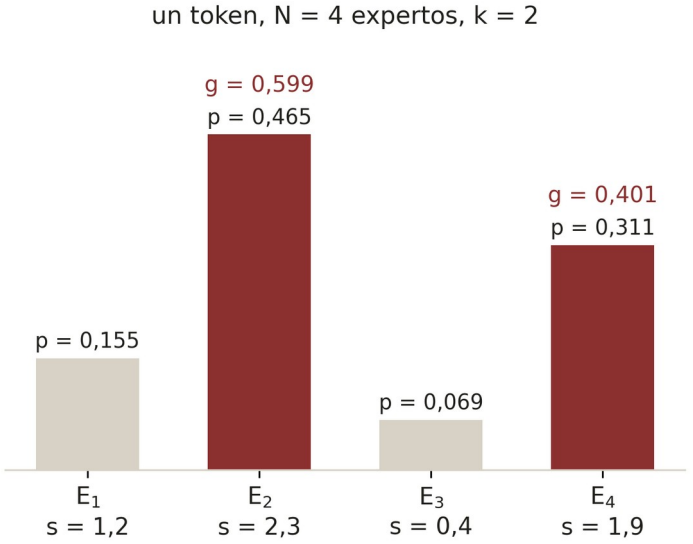
router: $p_e(x) \geq 0$ es el peso del experto e para x; suman 1

$$f(x) = \sum_{e=1}^{N} p_e(x)\, E_e(x)$$

densa: se evalúan los N expertos

$$f(x) = \sum_{e \in \mathcal{T}(x)} g_e(x)\, E_e(x), \qquad g_e = \frac{p_e}{\sum_{e' \in \mathcal{T}(x)} p_{e'}}$$

dispersa: solo los k con mayor p_e, con sus pesos renormalizados



Valores ilustrativos.

Router

$s_i = W_r\, z_i$ da un puntaje por experto; la softmax lo convierte en probabilidades p.

Selección

Se eligen E_2 y E_4, con p = 0,465 y 0,311. E_1 y E_3 no se evalúan.

Pesos

g = 0,465 / 0,776 = 0,599 y 0,401. La salida es f = 0,599 $E_2(z)$ + 0,401 $E_4(z)$.

Switch transformer: un experto por token

Con k = 1 cada token pasa por un solo experto. El cómputo por token es el de un transformer denso del mismo ancho, con N veces más parámetros en las redes por posición.

$$p_e(z) = \frac{e^{s_e}}{\sum_{j=1}^{N} e^{s_j}}, \qquad s = W_r\, z \qquad \text{probabilidad de cada experto}$$

$$e^* = \arg\max_e\, p_e(z) \qquad \text{un solo experto por token}$$

$$f = p_{e^*}(z)\, E_{e^*}(z) \qquad \text{la salida se multiplica por la probabilidad del elegido}$$

Por qué multiplicar por p

Si la salida fuera solo $E_{e^*}(z)$, el router no recibiría gradiente: el argmax no es derivable. La probabilidad p_{e^*} lo conecta con la pérdida.

Resultado

Con 64 expertos, el switch transformer alcanza la calidad de T5-Base en un séptimo del tiempo de preentrenamiento.

Escala

Switch-C tiene 1,6 billones de parámetros y 2.048 expertos.

Fedus, Zoph y Shazeer, JMLR 2022; Raffel et al., JMLR 2020 (T5).

Top-k: varios expertos por token

Con k > 1, cada token combina varios expertos. Shazeer agrega ruido a los puntajes al entrenar, para que el router explore expertos distintos.

$$H(z)_e = (W_g z)_e + \varepsilon \cdot \mathrm{Softplus}\big((W_{\text{ruido}}\, z)_e\big), \quad \varepsilon \sim \mathcal{N}(0,1)$$

el ruido reparte la carga al entrenar

$$\mathrm{KeepTopK}(v,k)_e = \begin{cases} v_e & \text{si } v_e \text{ está entre los } k \text{ mayores} \\ -\infty & \text{si no} \end{cases}$$

$$G(z) = \mathrm{softmax}\big(\mathbf{KeepTopK}(H(z),k)\big)$$

los no elegidos quedan con peso 0

KeepTopK

Deja los k puntajes mayores y pone −∞ en el resto. Tras la softmax, los no elegidos quedan con peso exactamente 0.

Ruido

Un término aleatorio, con magnitud aprendida, sobre cada puntaje. Ayuda a repartir la carga entre expertos durante el entrenamiento.

En los modelos

GShard y Mixtral usan k = 2; Qwen3 y DeepSeek-V3, k = 8 entre 128 y 256 expertos.

Shazeer et al., ICLR 2017; Lepikhin et al., ICLR 2021; Jiang et al., 2024.

Capacidad de los expertos

Cada experto procesa a lo más C tokens por lote. Si el router le envía más, los que sobran no pasan por ningún experto y siguen solo por el residual.

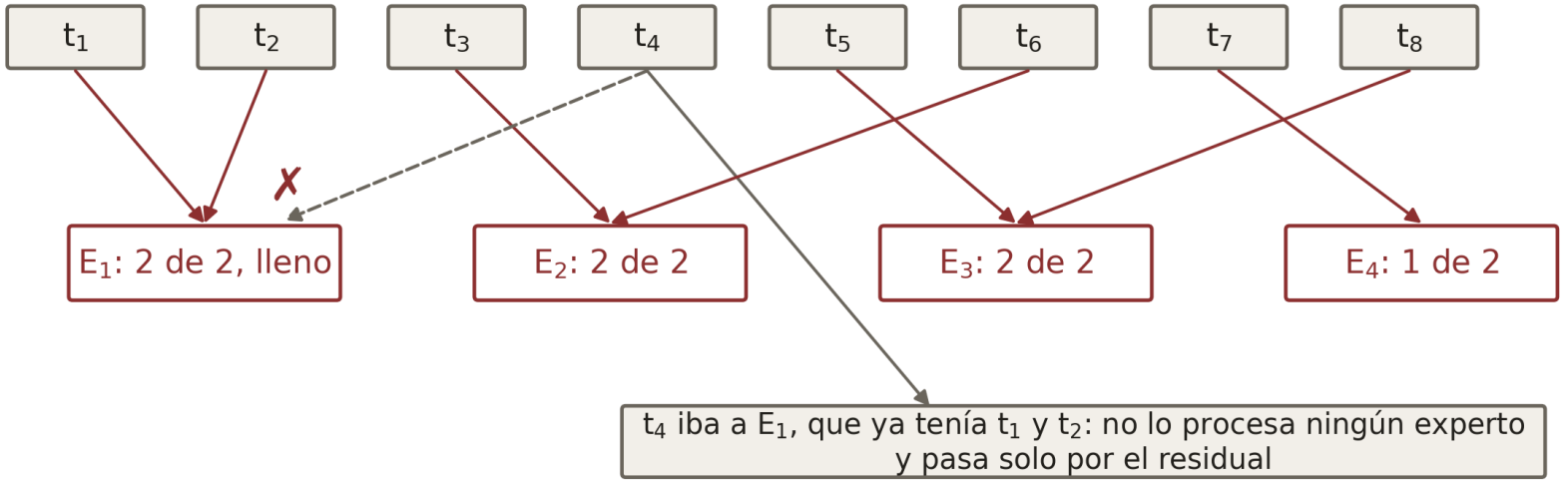
$$C = \frac{T}{N} \cdot \mathbf{cf}$$

T: tokens del lote; N: expertos; cf: factor de capacidad

$$y_i = \mathrm{LayerNorm}(z_i)$$

si el token excede la capacidad: solo el residual

T = 8 tokens, N = 4 expertos, cf = 1 → C = 2 tokens por experto



Fedus, Zoph y Shazeer, JMLR 2022; Lepikhin et al., ICLR 2021. Ejemplo ilustrativo.

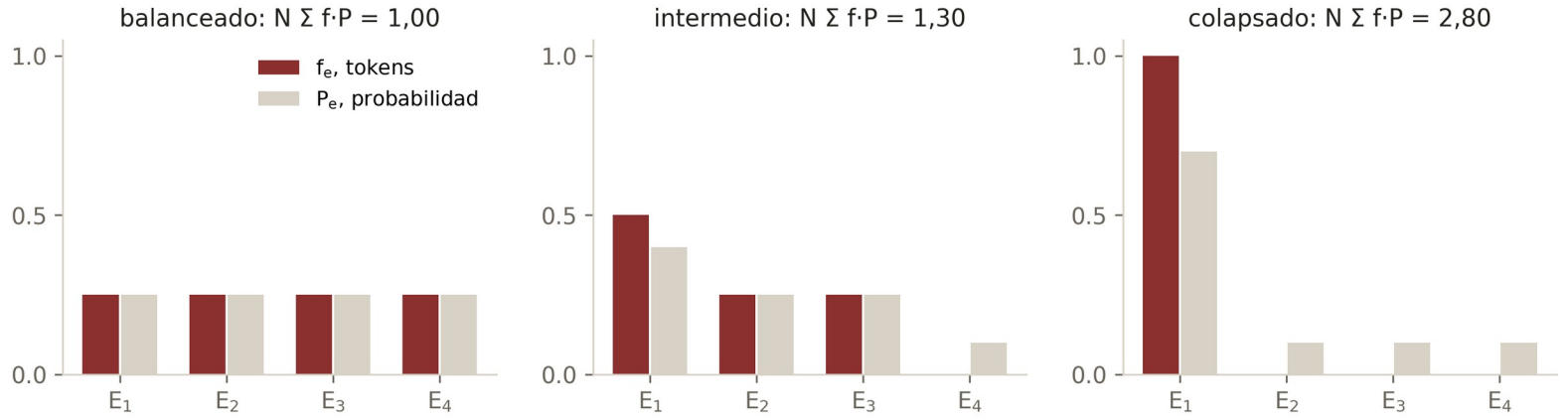
La pérdida de balance

Sin un término que lo impida, el router colapsa en pocos expertos. La pérdida de balance es mínima cuando los tokens y la probabilidad se reparten parejo.

$$f_e = \frac{1}{T}\sum_{t=1}^{T} \mathbf{1}\{e_t^* = e\} \qquad \text{fracción de tokens enviados al experto } e$$

$$P_e = \frac{1}{T}\sum_{t=1}^{T} p_e(z_t) \qquad \text{fracción de la probabilidad del router}$$

$$\mathcal{L}_{\text{bal}} = N \sum_{e=1}^{N} f_e \, P_e \qquad \text{mínima, igual a 1, con } f_e{=}P_e{=}1/N$$



Fedus, Zoph y Shazeer, JMLR 2022. Valores ilustrativos en la figura.

La función de pérdida completa

Un MoE se preentrena con next token prediction más dos términos auxiliares con pesos pequeños: el balance de carga y la z-loss, que estabiliza el router.

$$\mathcal{L}_z = \frac{1}{T}\sum_{t=1}^{T}\left(\log\sum_{j=1}^{N} e^{s_{t,j}}\right)^2 \qquad s_{t,j}\text{: puntaje del router para el token } t \text{ y el experto } j$$

$$\mathcal{L} = \mathcal{L}_{\text{NTP}} + \alpha\,\mathcal{L}_{\text{bal}} + c_z\,\mathcal{L}_z \qquad \alpha{=}10^{-2} \text{ en el switch transformer, } c_z{=}10^{-3} \text{ en ST-MoE}$$

z-loss

Es el cuadrado del log-sum-exp de los puntajes del router. Penaliza puntajes grandes: para el token del ejemplo vale 9,40.

Por qué

Puntajes grandes en 16 bits producen errores de redondeo en la exponencial y el entrenamiento diverge.

Los pesos α y c_z

Son chicos: el balance y la estabilidad son restricciones, no el objetivo. El objetivo sigue siendo predecir el siguiente token.

ST-MoE

Stable and Transferable MoE: un MoE que entrena sin divergir (stable) y se ajusta bien a tareas nuevas (transferable). La z-loss es su aporte a lo primero.

Fedus, Zoph y Shazeer, JMLR 2022; Zoph et al., 2022 (ST-MoE: Designing Stable and Transferable Sparse Expert Models).

LLMs con Mixture of Experts

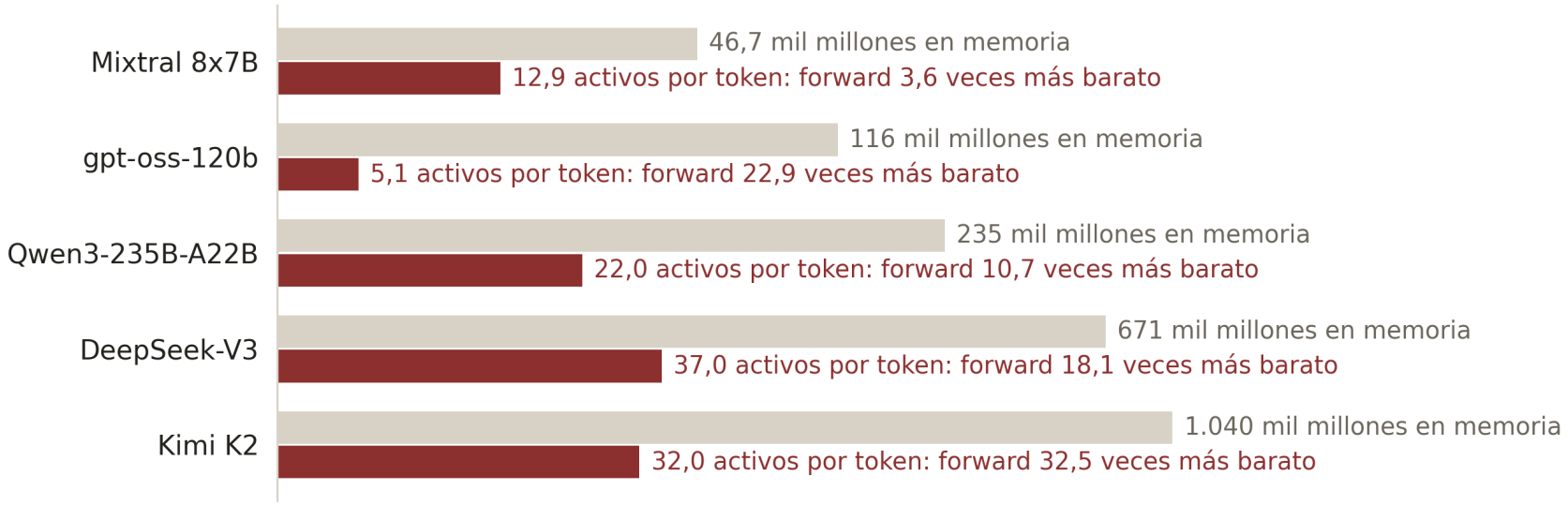
Los modelos abiertos más grandes de 2025 y 2026 son MoE. Los activos son los parámetros que usa cada token; los compartidos, expertos que procesan todos los tokens.

Modelo	Total	Activos	Expertos	k	Compartidos
Mixtral 8x7B (2024)	46.700 M	12.900 M	8	2	0
DeepSeek-V3 (2024)	671.000 M	37.000 M	256	8	1
Qwen3-235B-A22B (2025)	235.000 M	22.000 M	128	8	0
Llama 4 Maverick (2025)	400.000 M	17.000 M	128	1	1
OLMoE (2025)	6.900 M	1.300 M	64	8	0
gpt-oss-120b (2025)	116.800 M	5.100 M	128	4	0
Kimi K2 (2025)	1.040.000 M	32.000 M	384	8	1

M: millones. Jiang et al., 2024; DeepSeek-AI, 2024; Qwen Team, 2025; Meta, 2025; Muennighoff et al., ICLR 2025; OpenAI, 2025; Kimi Team, 2025.

El costo real de un MoE

Un MoE ocupa la memoria de todos sus parámetros, porque cualquier token puede necesitar cualquier experto. Pero cada token pasa solo por los activos, y el forward cuesta lo que un modelo denso de ese tamaño.



Jiang et al., 2024; OpenAI, 2025; Qwen Team, 2025; DeepSeek-AI, 2024; Kimi Team, 2025.